

Alocação dinâmica de memória

Quando o tamanho só se sabe rodando: `malloc`, `calloc`, `realloc` e `free`

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

Por que MAX_CONTATOS é uma decisão que você não deveria tomar



malloc e free: pedir um bloco e devolvê-lo



calloc: o mesmo pedido, com garantia de zeros



realloc: mudar de ideia sobre o tamanho



As quatro falhas clássicas — e como o compilador as denuncia

Hoje o ponteiro deixa de apontar para o vetor de outra pessoa: ele aponta para memória que *você* pediu.

**Por que alocar em tempo de
execução**

O que fizemos até agora

```
char nomes[4][20];
```

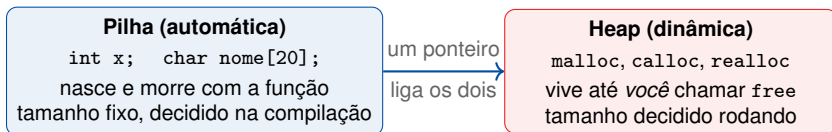
O tamanho é decidido por você, ao escrever o programa. O compilador reserva o espaço antes de o programa rodar.

Os dois modos de errar

Pequeno demais: chega o 5º contato e não há para onde ir.

Grande demais: `nomes[10000][20]` reserva 200 KB para guardar três nomes.

Quem sabe o tamanho certo é o *usuário*, enquanto o programa roda.



A regra que muda tudo

Na pilha, quem cria também destrói: a função termina, o espaço some. Na heap, o espaço permanece até você devolvê-lo — e permanece *mesmo* que você perca o ponteiro para ele.

```
1  #include <stdio.h>
2  #include <stdlib.h>  /* malloc, calloc, realloc, free */
3  #include <string.h>
4
5  int main(void) {
6      /* o que vem dentro de um bloco recém-alocado? */
7      int *sujo = malloc(4 * sizeof(int));
8      for (int i = 0; i < 4; i++)
9          sujo[i] = 777;
10     free(sujo);
11     int *a = malloc(4 * sizeof(int));
12     int *b = calloc(4, sizeof(int));
13     printf("A: malloc -> %d %d %d %d\n", a[0], a[1], a[2], a[3]);
14     printf("B: calloc -> %d %d %d %d\n", b[0], b[1], b[2], b[3]);
15     free(a);
16     free(b);
```

```
18     int capacidade = 2, n = 0;
19     char **nomes = malloc(capacidade * sizeof(char *));
20     int *idades = malloc(capacidade * sizeof(int));
21     if (nomes == NULL || idades == NULL) {
22         printf("sem memoria\n");
23         return 1;
24     }
25
26     const char *entrada[] = {"ana", "bruno", "carla",
27                               "davi", "elis"};
28     int idade[] = {23, 19, 31, 24, 28};
```

```
30     for (int i = 0; i < 5; i++) {
31         if (n == capacidade) {
32             int nova = capacidade * 2;
33             char **n2 = realloc(nomes, nova * sizeof(char *));
34             int *i2 = realloc(idades, nova * sizeof(int));
35             if (n2 == NULL || i2 == NULL) { return 1; }
36             printf("C: cheia com %d -> cresce para %d | nomes: %s\n",
37                 capacidade, nova,
38                 (n2 == nomes) ? "mesmo bloco" : "bloco NOVO");
39             nomes = n2; idades = i2; capacidade = nova;
40         }
41         nomes[n] = malloc(strlen(entrada[i]) + 1);
42         strcpy(nomes[n], entrada[i]);
43         idades[n] = idade[i];
44         n++;
45     }
```

```
47     printf("D: %d contatos guardados em capacidade %d\n",
48           n, capacidade);
49     for (int i = 0; i < n; i++)
50         printf("  %-6s %3d\n", nomes[i], idades[i]);
51     printf("E: sizeof(nomes) = %zu | strlen(nomes[1]) + 1 = %zu\n",
52           sizeof(nomes), strlen(nomes[1]) + 1);
53
54     for (int i = 0; i < n; i++)
55         free(nomes[i]);
56     free(nomes);
57     free(idades);
58     return 0;
59 }
```

Antes de compilar, escreva sua resposta

1. Na linha **A**, o que sai de um bloco de `malloc` que ninguém preencheu ainda?
2. Na linha **B**, o que muda por ter usado `calloc`?
3. Quantas vezes a linha **C** aparece, se a capacidade começa em 2 e chegam 5 contatos?
4. Na linha **C**, o vetor continua no *mesmo bloco* ou vai para um *bloco novo*?
5. Na linha **E**, quanto vale `sizeof(nomes)`, sendo `nomes` uma agenda de 8 posições?

```
$ gcc -Wall -Wextra agenda.c -o agenda
$ ./agenda
A: malloc -> 0 0 0 0
B: calloc -> 0 0 0 0
C: cheia com 2 -> cresce para 4 | nomes: bloco NOVO
C: cheia com 4 -> cresce para 8 | nomes: bloco NOVO
D: 5 contatos guardados em capacidade 8
   ana      23
   bruno    19
   carla    31
   davi     24
   elis     28
E: sizeof(nomes) = 8 | strlen(nomes[1]) + 1 = 6
```

malloc **e** free

```
void *malloc(size_t n);
```

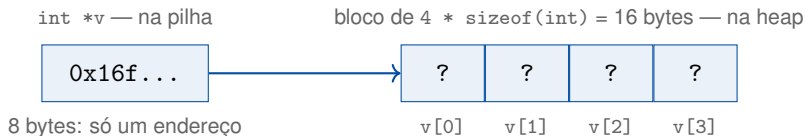
Pede ao sistema um bloco de **n bytes** contíguos na heap e devolve o endereço do primeiro byte — ou NULL se não houver memória.

Intuição

`malloc` não sabe o que você vai guardar: ele conta *bytes*, não elementos. Por isso o pedido quase sempre tem a forma `quantidade * sizeof(tipo)`.

```
int *v = malloc(100 * sizeof(int)); — 100 inteiros
```

```
int *v = malloc(100); — 100 bytes: 25 inteiros
```



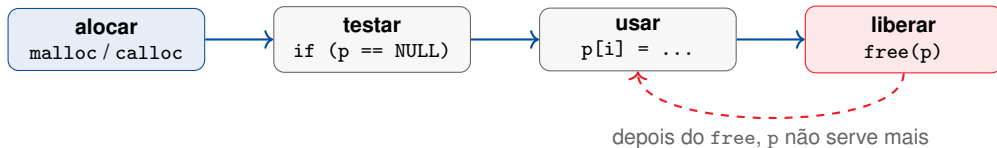
Duas coisas separadas

O ponteiro é uma variável comum, na pilha, que morre com a função. O bloco é outra coisa, na heap, que **não** morre com a função. Perder o ponteiro não apaga o bloco — só torna impossível devolvê-lo.

```
1  int *v = malloc(n * sizeof(int));
2  if (v == NULL) {                      /* o pedido pode falhar */
3      printf("sem memoria\n");
4      return 1;
5  }
6  /* daqui para baixo, v e' um vetor de n inteiros */
```

Por que isso importa

Se `malloc` falha e você não testa, `v[0] = 3` escreve no endereço 0: o programa morre com *segmentation fault* em um lugar que nada tem a ver com a causa.



Um free para cada malloc

Cada chamada bem-sucedida de malloc, calloc ou realloc corresponde a exatamente uma chamada de free. Nem zero, nem duas.

Faz

- devolve o bloco ao sistema, que pode reaproveitá-lo no próximo `malloc`;
- aceita `free(NULL)` sem reclamar.

Não faz

- não altera o ponteiro: `p` continua com o mesmo endereço, agora inválido;
- não apaga o conteúdo: os bytes antigos podem continuar lá;
- não avisa você se esquecer de chamá-lo.

Hábito barato

`free(p); p = NULL;` — um ponteiro nulo falha alto e cedo; um ponteiro pendurado falha baixo e tarde.

O que aprendemos

- `malloc(n * sizeof(tipo))` devolve o endereço de um bloco de bytes na heap, ou `NULL`;
- o ponteiro vive na pilha; o bloco vive na heap e sobrevive à função;
- testar o `NULL` é parte da alocação, não um extra;
- `free` devolve o bloco, mas não mexe no seu ponteiro.

calloc

O que o programa fez

Preencheu um bloco com 777, liberou o bloco e pediu outro do mesmo tamanho — que, muito provavelmente, é o mesmo bloco de volta.

A resposta depende da máquina:

- nesta aqui: 0 0 0 0
- em muitos Linux: 777 777 777 777
- com MallocScribble=1: -1431655766

Três respostas para o mesmo código.

Isso não é um detalhe de implementação curioso: é a definição de *valor indeterminado*.

O conteúdo de um bloco de `malloc` é indeterminado

Ler `v[i]` antes de escrever nele é comportamento indefinido. Se hoje saiu zero, foi coincidência: o sistema entregou uma página limpa ou um bloco que por acaso estava zerado. Amanhã, com outra entrada, sai outra coisa.

Consequência prática

Um contador que começa em `v[i]++` sem inicialização passa nos seus testes e falha na máquina do professor. É exatamente o tipo de bug que só aparece longe de quem o escreveu.

```
void *calloc(size_t quantidade, size_t tamanho);
```

Aloca espaço para quantidade elementos de tamanho bytes cada e **preenche tudo com zeros** antes de devolver o endereço.

	malloc	calloc
argumentos	1: total de bytes	2: quantidade e tamanho
conteúdo	indeterminado	todos os bytes em zero
custo	só reserva	reserva e limpa
$n * sz$ transborda	silenciosamente	detectado, devolve NULL

```
1  /* vou preencher tudo agora mesmo: zerar seria trabalho jogado fora */
2  int *copia = malloc(n * sizeof(int));
3  for (int i = 0; i < n; i++) copia[i] = origem[i];
4
5  /* vou acumular: preciso começar do zero */
6  int *contagem = calloc(256, sizeof(int));
7  for (int i = 0; texto[i] != '\0'; i++) contagem[(unsigned char) texto[i]]++;
```

Regra de bolso

Se a primeira coisa que você faria é um laço escrevendo zeros, use `calloc`. Se a primeira coisa é um laço escrevendo os valores de verdade, use `malloc`.

O que aprendemos

- `malloc` entrega bytes com conteúdo indeterminado — zeros observados são coincidência, não contrato;
- `calloc(q, t)` entrega $q * t$ bytes zerados e ainda protege contra o transbordo da multiplicação;
- zerar custa tempo: escolha pelo que você vai fazer com o bloco, não por hábito.

realloc

A saída

```
C: cheia com 2 -> cresce para 4 | nomes:  bloco NOVO
```

```
C: cheia com 4 -> cresce para 8 | nomes:  bloco NOVO
```

A pergunta

Se o bloco é novo, o que aconteceu com os nomes que já estavam lá?
E com o bloco antigo?

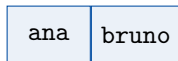
A pista

Os cinco contatos aparecem certos no final. Ninguém copiou nada explicitamente no seu código.

```
void *realloc(void *p, size_t novos_bytes);
```

Redimensiona o bloco apontado por `p` para `novos_bytes` e devolve o endereço do bloco resultante — que pode ser o mesmo ou **outro**. O conteúdo antigo é preservado até o menor dos dois tamanhos.

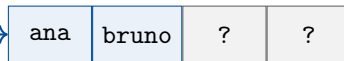
bloco antigo (2 posições)



copia

e libera o antigo

bloco novo (4 posições)



O espaço acrescentado **não** é zerado: é indeterminado, como no `malloc`.

```
1  /* ERRADO: se realloc falhar, devolve NULL e o endereço  
2    do bloco antigo -- que continua alocado -- se perde. */  
3  v = realloc(v, nova * sizeof(int));  
4  
5  /* CERTO: só troca v depois de saber que deu certo. */  
6  int *tmp = realloc(v, nova * sizeof(int));  
7  if (tmp == NULL) { /* v ainda vale: dá para se recuperar */ }  
8  v = tmp;
```

Duas coisas de uma vez

A versão errada vazava memória e destrói a única cópia do ponteiro para os dados que você ainda tinha. Foi por isso que agenda.c usou n2 e i2.

```
1  int *v = malloc(2 * sizeof(int));
2  v[0] = 10;
3  int *w = realloc(v, 100 * sizeof(int));
4  printf("w[0] = %d\n", w[0]);    /* 10: o conteudo veio junto */
5  printf("v[0] = %d\n", v[0]);    /* v pode ja' ter sido liberado */
```

O que o AddressSanitizer diz da última linha

```
ERROR: AddressSanitizer: heap-use-after-free on address 0x6020000000f0
      #0 in main rtrap.c:8
```

Depois de um `realloc` bem-sucedido, existe *um* ponteiro válido: o que ele devolveu.

Chamada	Efeito	Equivale a
<code>realloc(NULL, n)</code>	aloca do zero	<code>malloc(n)</code>
<code>realloc(p, maior)</code>	cresce, copiando o conteúdo	—
<code>realloc(p, menor)</code>	encolhe, descartando o fim	—

Por que dobrar a capacidade, e não somar 1?

Crescer de 1 em 1 até n pode copiar da ordem de n^2 elementos. Dobrando, o total copiado fica da ordem de n : cada elemento é movido, em média, um número constante de vezes. Por isso a agenda foi 2, 4, 8.

O que aprendemos

- `realloc` pode devolver um endereço diferente — e frequentemente devolve;
- ele copia o conteúdo antigo e libera o bloco antigo por você;
- o espaço novo não vem zerado;
- receba o resultado em um ponteiro temporário e só então substitua o original;
- depois da troca, o ponteiro antigo está morto.

Armadilhas e diagnóstico

Falha	O que é	Sintoma típico
Vazamento	alocar e nunca liberar	programa incha com o tempo
Ponteiro pendurado	usar depois do <code>free</code>	lixo ou <i>segfault</i> aleatório
<code>free</code> duplo	liberar duas vezes	aborta em <code>free</code> , sem pista
Estouro do bloco	escrever além do tamanho	corrompe o vizinho

O que as quatro têm em comum

Nenhuma delas falha onde foi cometida. O programa quebra depois, em outro lugar, com uma mensagem que aponta para código inocente.

```
1 void carrega_agenda(void) {
2     char **nomes = malloc(5 * sizeof(char *));
3     for (int i = 0; i < 5; i++) {
4         nomes[i] = malloc(strlen(entrada[i]) + 1);
5         strcpy(nomes[i], entrada[i]);
6     }
7     free(nomes);    /* liberou o vetor, nao os cinco nomes */
8 }
```

Um free não basta

nomes guarda cinco endereços. Liberar nomes devolve as 40 bytes dos *ponteiros* e torna os cinco blocos de texto inalcançáveis. Libere de dentro para fora: primeiro cada `nomes[i]`, depois `nomes`.

```
# macOS
```

```
$ MallocStackLogging=1 leaks -atExit -- ./vaza
```

```
Process 3103: 5 leaks for 80 total leaked bytes.
```

```
STACK OF 5 INSTANCES OF 'ROOT LEAK: <malloc in carrega_agenda>':
```

```
1 (16 bytes) ROOT LEAK: <malloc in carrega_agenda 0xc8100c000>
```

```
# Linux
```

```
$ gcc -g -fsanitize=address vaza.c -o vaza && ./vaza
```

```
$ valgrind --leak-check=full ./vaza
```

A ferramenta aponta a *função onde o bloco foi alocado* — e é isso que transforma um vazamento em um conserto de duas linhas.

```
$ gcc -Wall -g -fsanitize=address uaf.c -o uaf && ./uaf
ERROR: AddressSanitizer: heap-use-after-free on address 0x6020000000f0
READ of size 2 at 0x6020000000f0 thread T0
    #0 in main uaf.c:8
0x6020000000f0 is located 0 bytes inside of 6-byte region
freed by thread T0 here:
    #0 in free
    #1 in main uaf.c:7
```

Leia a mensagem inteira

Ela diz três coisas: *o que* (uso após liberação), *onde* (linha 8) e *por causa de quem* (o free da linha 7). Um *segfault* comum diria apenas “o programa morreu”.

Checklist

1. pedir quantidade * sizeof(tipo), nunca um número solto;
2. testar NULL logo depois de alocar;
3. receber realloc em um ponteiro temporário;
4. liberar de dentro para fora, e uma vez só;
5. `p = NULL` depois do `free`, e compilar com `-Wall -g -fsanitize=address` enquanto desenvolve.

O que aprendemos

- as falhas de memória não quebram o programa onde foram cometidas;
- vaziar é perder o endereço, não perder os dados — o bloco continua ocupado;
- quem alocou em camadas libera em camadas;
- as ferramentas transformam “às vezes dá erro” em “linha 8, por causa da linha 7”.

Mãos à obra

Tarefa 1

Troque a capacidade inicial de 2 para 1 e o fator de crescimento de $\times 2$ para $+1$. Quantas vezes a linha **C** passa a aparecer? Explique o número.

Tarefa 2

Troque o `malloc` de `idades` por `calloc` e imprima `idades[7]` logo depois de alocar, antes de preencher. O que muda em relação ao `malloc`?

Tarefa 3

Comente o laço que libera `nomes[i]` e rode o detector de vazamentos. Quantos blocos ele acusa? Por que esse número?

Uma agenda que também remove

Escreva `remover(char **nomes, int *idades, int n, const char *quem)` que apaga um contato e devolve a nova quantidade.

Requisitos:

- libere o nome removido antes de perder o endereço dele;
- feche o buraco deslocando os contatos seguintes;
- se a agenda ficar com menos de um quarto da capacidade, encolha o vetor pela metade com `realloc`;
- o programa deve terminar sem vazamentos e sem erros do sanitizer.

O que mudou hoje

O tamanho das suas estruturas deixou de ser uma decisão de quem escreve o programa e passou a ser uma decisão de quem o executa.

O preço

Você ganhou o controle do tempo de vida da memória — e junto com ele a obrigação de devolvê-la, exatamente uma vez, no lugar certo.

Para levar

- `malloc`: bytes, sem garantia de conteúdo, pode falhar;
- `calloc`: bytes zerados, dois argumentos;
- `realloc`: novo tamanho, endereço possivelmente novo;
- `free`: uma vez, de dentro para fora.

-  André Backes.
Linguagem C: completa e descomplicada.
Elsevier, Rio de Janeiro, 2013.
-  Randal E. Bryant and David R. O'Hallaron.
Computer Systems: A Programmer's Perspective.
Pearson, Boston, 3 edition, 2015.
Seção 3.9: alinhamento de dados e layout de structs.
-  Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Algoritmos: teoria e prática.
Elsevier, Rio de Janeiro, 3 edition, 2012.

-  **Paul Deitel and Harvey Deitel.**
C: como programar.
Pearson Prentice Hall, São Paulo, 6 edition, 2011.
-  **Ulrich Drepper.**
What Every Programmer Should Know About Memory.
Red Hat, Inc., 2007.
Seção 6.2: efeito da ordem de percurso de matrizes sobre a cache.
-  **Ulrich Drepper.**
How to write shared libraries.
Documento técnico, 2011.
Versão 4.1.2.



Paulo Feofiloff.

Algoritmos em linguagem C.

Elsevier, Rio de Janeiro, 2009.



Free Software Foundation.

Gnu coding standards.

Free Software Foundation.

Acesso em 2026.



Free Software Foundation.

Using the GNU compiler collection — options to request or suppress warnings.

Manual do GCC.

Acesso em 2026.



ISO/IEC.

Iso/iec 9899:2018 — information technology: Programming languages: C. Technical report, International Organization for Standardization, 2018.

Seção 6.7.2.1: structure and union specifiers.

-  Brian W. Kernighan and Rob Pike.
The Practice of Programming.
Addison-Wesley, Reading, MA, 1999.
Cap. 1, “Style”: nomes, expressões, consistência, números mágicos e comentários.
-  Brian W. Kernighan and Dennis M. Ritchie.
C: a linguagem de programação — padrão ANSI.
Campus, Rio de Janeiro, 1989.

-  K. N. King.
C Programming: A Modern Approach.
W. W. Norton, New York, 2 edition, 2008.
Capítulo 16: estruturas, uniões e enumerações.

-  Steve McConnell.
Code Complete: A Practical Handbook of Software Construction.
Microsoft Press, Redmond, WA, 2 edition, 2004.
Cap. 11 (nomes de variáveis), cap. 31 (layout e estilo) e cap. 32 (código autodocumentado).

-  Robert Mecklenburg.
Managing Projects with GNU Make.
O'Reilly Media, 3 edition, 2004.
-  Herbert Schildt.
C completo e total.
Makron Books, São Paulo, 3 edition, 1997.
-  Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov.
AddressSanitizer: A fast address sanity checker.
In *USENIX Annual Technical Conference (ATC)*, pages 309–318, 2012.
Ferramenta usada no tutorial para detectar vazamentos e uso após free.

-  Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.
Estruturas de dados usando C.
Pearson Makron Books, São Paulo, 1995.
-  The Linux Kernel Developers.
Linux kernel coding style.
Documentação do kernel Linux, [Documentation/process/coding-style.rst](#).
Acesso em 2026.
-  The LLVM Project.
Clang-format style options.
Documentação do Clang.
Acesso em 2026.

-  Paul R. Wilson, Mark S. Johnstone, Michael Neely, and David Boles.
Dynamic storage allocation: A survey and critical review.
In *International Workshop on Memory Management (IWMM)*, pages 1–116.
Springer, 1995.
Panorama de estratégias de alocação dinâmica e fragmentação.
-  Nivio Ziviani.
Projeto de algoritmos: com implementações em Pascal e C.
Cengage Learning, São Paulo, 3 edition, 2011.